

Transformer Architecture: A Conceptual Overview (Demo)

Introduction

The transformer architecture, introduced in the mid-2010s, replaced recurrent and convolutional sequence models with a design built entirely around attention mechanisms. Instead of processing tokens one at a time in sequence, a transformer represents an entire sequence at once and lets every position attend to every other position directly. This removed the sequential bottleneck that limited how well recurrent networks could be parallelized during training.

This document, prepared as demo knowledge-base content, summarizes the core components of the transformer block: self-attention, multi-head attention, positional encoding, and the feed-forward sublayer, along with why each component exists and what problem it solves.

Self-Attention

Self-attention computes, for each token, a weighted combination of all other tokens' representations, where the weights are learned based on how relevant each other token is to the current one. Each token is projected into three vectors: a query, a key, and a value. The attention weight between two tokens is computed from the dot product of one token's query and the other's key, scaled and passed through a softmax so the weights sum to one across all tokens.

This mechanism allows a model to directly relate a pronoun to the noun it refers to, or a closing bracket to its matching opening bracket, regardless of how far apart they are in the sequence — something recurrent models struggled to do reliably over long distances because information had to pass through every intermediate step.

Multi-Head Attention

Rather than computing a single attention distribution, transformers compute several attention 'heads' in parallel, each with its own learned query, key, and value projections. Each head can specialize in a different kind of relationship — one head might track syntactic dependencies, another might track coreference. The outputs of all heads are concatenated and linearly projected back to the model's hidden size.

Empirically, multi-head attention tends to produce richer representations than a single large attention head with the same total parameter count, because different heads are free to specialize rather than averaging together many kinds of relationships into one distribution.

Positional Encoding

Because self-attention has no inherent notion of token order — it treats the input as a set, not a sequence — transformers add positional information explicitly. The original design used fixed sinusoidal functions of position, added to each token's embedding before the first layer; later architectures often use learned positional embeddings or relative position encodings computed inside the attention mechanism itself.

Without positional encoding, the sentences 'the dog bit the man' and 'the man bit the dog' would be indistinguishable to a pure self-attention layer, since both contain the same set of tokens.

Feed-Forward Sublayer and Residual Connections

Each transformer block also contains a position-wise feed-forward network, applied identically to every token, typically consisting of two linear layers with a non-linearity in between. Residual connections wrap both the attention sublayer and the feed-forward sublayer, and layer normalization is applied to stabilize training in deep stacks of these blocks.

Stacking many such blocks — each combining attention (which mixes information across positions) with a feed-forward layer (which transforms each position independently) — is what gives transformers their representational depth.